

Analysis of Algorithms

Lecture 1

Course Teacher: Miss Shagufta Iftikhar

Lectures Outline

- Introduction to Course
- Role of Algorithms
- Algorithm and its Attributes
- Analyzing algorithms
- Approaches to Analysis: Empirical Approach, Analytical
- Algorithm Specification
- RAM Computational Model
- Computational Complexity

- This course is an advanced undergraduate course focusing on the design and analysis of algorithms. It covers fundamental concepts such as the role of algorithms in computing, asymptotic analysis (Big-O, Big- Ω , Big- Θ), recursion and recurrence relations, and loop invariants, study of sorting and searching algorithms, string matching algorithms, heaps and hashing, graph traversal algorithms, and major algorithm design techniques including divide and conquer, greedy algorithms, and dynamic programming.

- Course Code:
 - CS: CSAA-341, AI: CSAA-226
- Credit Hours: [3+0]
- Prerequisite:
 - CS: [CSDS-208], AI: [CSDS-224]
- Text books:

Introduction to Algorithms (3rd edition) by Thomas H. Corman, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein

Algorithm Design (1st edition, 2013/2014) Jon Kleinberg, Eva Tardos

Why Study Algorithms?

- Multiple solutions exist for the same problem
- All correct algorithms are not equally efficient
- Efficiency matters for large inputs
- Real-world systems depend on fast algorithms

- Core of problem solving
- Used in:
 - Data processing
 - Networking
 - Artificial Intelligence
 - Operating Systems
- Determines performance and feasibility

What is an Algorithm?

An algorithm is a **well-defined** and **effective** sequence of computation steps that takes some value, or set of values, as input and produces some value, or set of values, as output.

- Input: A sequence of n numbers $\langle a_1, a_2, a_3 \dots a_n \rangle$
- Output: A permutation (re-ordering) $\langle b_1, b_2, b_3 \dots b_n \rangle$ of the input sequence such that $b_1 < b_2 < b_3 \dots < b_n$

- Input
- Output
- Definiteness
- Finiteness
- Effectiveness
- Correctness

- Sorting/searching are by no mean the only computational problem for which algorithms have been developed.
- Otherwise, we wouldn't have the whole course on this topic
- Practical application of algorithms are ubiquitous and include the following examples

- Internet world
- Electronic commerce
- Manufacturing and other commercial settings
- Shortest path
- Matrices multiplication order
- DNA sequence matching

- There are many candidate solutions, most of which are not what we want, finding one that we do want can present quite a challenge.
- There are practical applications (its not just mathematical exercises to develop algorithms.)

- Predict performance before implementation
- Compare multiple algorithms
- Algorithms help us to understand **scalability**.
- Optimize resource usage
- Performance often draws the line between what is feasible and what is impossible.
- Algorithmic mathematics provides a **language** for talking about program behavior.
- Performance is the **currency** of computing.
- Speed is fun!

What Does Algorithm Analysis Measure?

- Time complexity
- Space complexity
- Best case
- Average case
- Worst case

- Empirical Approach
- Analytical Approach

- Implement algorithm
- Run on test inputs
- Measure execution time

- Machine dependent
- Input dependent
- Not scalable for large inputs
- Hard to generalize results

- Uses mathematical modeling
- Independent of machine
- Measures growth rate
- Predicts performance for large inputs

- Description of algorithm logic
- Written using:
 - Pseudocode
 - Flowcharts
- Independent of programming language

- Random Access Machine (RAM)
- Assumes:
 - Constant time for basic operations
 - Sequential execution
- Simplifies analysis

- Measures algorithm efficiency
- Expressed as a function of input size
- Focus on growth rate
- Independent of hardware



**Thank
You**